

บทที่ 4

การเขียนโปรแกรมบนระบบปฏิบัติการวินโดวส์และการใช้งานไลบรารีต้นแบบมาตรฐาน

4.1 การเขียนโปรแกรมบนระบบปฏิบัติการวินโดวส์

การเขียนโปรแกรมให้ทำงานบนระบบปฏิบัติการวินโดวส์แตกต่างจากการเขียนโปรแกรมบนระบบปฏิบัติการดอส (DOS) อย่างมาก ในระบบปฏิบัติการดอสมีการทำงานของโปรแกรมได้เพียงโปรแกรมเดียวในขณะหนึ่ง หรือสามารถใช้เวลาการทำงานของโปรเซสเซอร์ (processing time) ทั้งหมดได้ และสามารถควบคุมอุปกรณ์ในเครื่องคอมพิวเตอร์ได้ทั้งหมด แต่ในขณะเดียวกันผู้เขียนโปรแกรมก็ต้องตรวจสอบอุปกรณ์ต่างๆ ที่มีอยู่ในเครื่องคอมพิวเตอร์เองด้วย ซึ่งเป็นงานที่ยากลำบากมากเนื่องจากอุปกรณ์คอมพิวเตอร์มีจำนวนมากมาย ในขณะที่จะการเขียนโปรแกรมบนระบบปฏิบัติการวินโดวส์จะต้องแบ่งการใช้งานโปรเซสเซอร์กับโปรแกรมอื่นๆ และไม่สามารถเข้าไปควบคุมการทำงานของอุปกรณ์ที่มีอยู่ในเครื่องได้โดยตรง แต่ระบบปฏิบัติการวินโดวส์จะช่วยให้เราเขียนโปรแกรมได้สะดวกขึ้นเนื่องจากระบบปฏิบัติการได้จัดการงานที่สลับซับซ้อนให้เราทั้งหมดแล้ว เช่น การตรวจสอบอุปกรณ์ที่มีอยู่ในเครื่อง เป็นต้น นอกจากนั้น การทำงานหลายๆ อย่างก็สะดวกขึ้น เช่น ถ้าเราต้องการเล่นเพลงก็ใช้คำสั่งเพียง PlaySound ในขณะที่การเขียนโปรแกรมบนระบบปฏิบัติการดอสจะต้องทำงานที่สลับซับซ้อนกว่านี้มาก โดยการเขียนโปรแกรมบนวินโดวส์จะต้องใช้ Win32 API (Application Programming Interface)

Win32 API คือ ชุดคำสั่งในภาษาซีที่เราเขียนโปรแกรมให้ทำงานบนระบบปฏิบัติการวินโดวส์ได้ ซึ่งได้จัดการงานที่มีสลับซับซ้อนหลายอย่างให้กับเราแล้ว เช่น การทำงานแบบมัลติทาสกิ้ง (multi-tasking), การป้องกันหน่วยความจำ (memory protection) เป็นต้น นอกจากนั้น ยังมีส่วนของอินเทอร์เฟซ (interface) ที่ช่วยในการแสดงผลแบบ GUI (Graphics User Interface) เช่น ไดอะล็อกบ็อกซ์ (dialog box), เมนู (menu) และงานเกี่ยวกับมัลติมีเดีย (multimedia) ซึ่งชุดคำสั่งเหล่านี้จะทำให้เราทำงานได้ง่ายขึ้นมาก

API ในระบบปฏิบัติการวินโดวส์มีหลายกลุ่มด้วยกัน และสำหรับงานแต่ละชนิดก็มีการทำงานแตกต่างกันในทีนี้จะนำเสนอแต่ละเฉพาะในส่วนที่มีความจำเป็นในการใช้งานเท่านั้น

4.1.1 ชนิดของข้อมูลใน Win32 API (Win32 Data Types)

นอกจากชนิดของข้อมูลที่มีอยู่ในภาษา C และ C++ แล้วใน Win32 API ยังได้กำหนดชนิดของข้อมูลขึ้นมาอีกจำนวนหนึ่ง ซึ่งสามารถนำไปใช้ในการกำหนดค่าส่งคืนของฟังก์ชัน (function return value), พารามิเตอร์ของฟังก์ชันและเมสเสจ (function and message parameter) และเป็นสมาชิกในข้อมูลอื่น (structure member) ซึ่งต่อไปนี้เป็นชื่อและขนาดของข้อมูลเหล่านี้บางส่วนที่นำมาใช้งาน

BOOL	ค่าตัวแปรบูลีน (มีค่าเป็น TRUE หรือ FALSE)
BOOLEAN	ค่าตัวแปรบูลีน (มีค่าเป็น TRUE หรือ FALSE)
BYTE	ไบต์ (byte) มีขนาด 8 บิต (bit)
CALLBACK	ระเบียบในการเรียกใช้สำหรับฟังก์ชันเรียกกลับ (calling convention for callback function)

CHAR	ตัวอักษรขนาด 8 บิต ในระบบปฏิบัติการวินโดวส์ ในตัวอักษรในระบบ ANSI
COLORREF	ค่าสีเขียว, ค่าสีฟ้า และค่าสีแดง (RGB) มีขนาด 32 บิต
DLGPROC	ค่าพอยเตอร์ (พอยเตอร์) ที่ชี้ไปยังโคออดิเนตบ็อกซ์ที่กำหนดไว้ในฟังก์ชันเรียกกลับ
DWORD	จำนวนเต็มแบบไม่คิดเครื่องหมายขนาด 32 บิต (32 bit unsigned integer)
FLOAT	จำนวนทศนิยมขนาด 32 บิต
HANDLE	ค่าแฮนเดิล (handle) ของวัตถุ
HBITMAP	ค่าแฮนเดิล (handle) ของบิตแมพ (bitmap)
HDC	ค่าแฮนเดิล (handle) ของดีไวซ์คอนเท็กซ์ (device context : DC)
HFILE	ค่าแฮนเดิล (handle) ของไฟล์
HFONT	ค่าแฮนเดิล (handle) ของฟอนต์ (font)
HICON	ค่าแฮนเดิล (handle) ของไอคอน (icon)
HINSTANCE	ค่าแฮนเดิล (handle) ของอินสแตนซ์ (instance)
HMENU	ค่าแฮนเดิล (handle) ของเมนู (menu)
HTREEITEM	ค่าแฮนเดิล (handle) ของไอเทม (item) ในทรีวิวคอนโทรล (tree-view control)
HWND	ค่าแฮนเดิล (handle) ของวินโดวส์
LONG	ค่าจำนวนเต็มแบบคิดเครื่องหมายขนาด 32 บิต (32-bit signed integer)
ONGLONG	ค่าจำนวนเต็มแบบคิดเครื่องหมายขนาด 64 บิต (64-bit signed integer)
LPARAM	ค่าพารามิเตอร์ของเมสเสจ (message parameter)
LPBOOL	ค่าพอยเตอร์ที่ชี้ไปยังข้อมูลชนิด BOOL
LPBYTE	ค่าพอยเตอร์ที่ชี้ไปยังข้อมูลชนิด BYTE
LPCTSTR	ค่าพอยเตอร์ที่ชี้ไปยังสตริงที่จบด้วยค่าศูนย์ (null-terminated string) ขนาด 8 บิต ตัวอักษรใช้แบบ ANSI
LPDWORD	ค่าพอยเตอร์ที่ชี้ไปยังข้อมูลชนิด DWORD
LPINT	ค่าพอยเตอร์ที่ชี้ไปยังข้อมูลชนิด INT
LPLONG	ค่าพอยเตอร์ที่ชี้ไปยังข้อมูลชนิด LONG
LPSTR	ค่าพอยเตอร์ที่ชี้ไปยังสตริงที่จบด้วยค่าศูนย์ (null-terminated string) ขนาด 8 บิต ตัวอักษรใช้แบบ ANSI
LPVOID	ค่าพอยเตอร์ที่ชี้ไปยังข้อมูลชนิดใดๆ
LPWORD	ค่าพอยเตอร์ที่ชี้ไปยังข้อมูลชนิด WORD
LRESULT	ผลลัพธ์ที่คิดเครื่องหมายสำหรับการประมวลผลเมสเสจ (signed result of message processing)
MSG	โครงสร้างของเมสเสจ

RECT	โครงสร้างของพื้นที่สี่เหลี่ยม
TCHAR	จะเป็น WCHAR ถ้ากำหนดว่าใช้ UNICODE ถ้าไม่ได้กำหนดจะเป็น CHAR
UINT	INT แบบไม่คิดเครื่องหมาย
ULONG	LONG แบบไม่คิดเครื่องหมาย
ULONGLONG	จำนวนเต็มแบบไม่คิดเครื่องหมายขนาด 64 บิต (64-bit unsigned integer)
VOID	ชนิดของข้อมูลใดๆ
WINAPI	ระเบียบในการเรียกใช้สำหรับ Win32 API (calling convention for Win32 API)
WNDCLASSEX	โครงสร้างของวินโดว์คลาส (Window class)
WNDPROC	ค่าพอยเตอร์ที่ชี้ไปยังวินโดว์โปรซีเจอร์ (Window procedure) ที่กำหนดไว้โดยโปรแกรม)
WORD	จำนวนเต็มแบบไม่คิดเครื่องหมายขนาด 16 บิต (16-bit unsigned integer)
LPARAM	ค่าพารามิเตอร์ของเมสเสจขนาด 32 บิต (32-bit message parameter)

4.1.2 การกำหนดค่าแบบฮังการี (Hungarian Notation)

ในการตั้งชื่อตัวแปรเพื่อให้ผู้ใช้งานต่อการจดจำชนิดข้อมูลของตัวแปรนั้นๆ มีรูปแบบในการกำหนดชื่อหลายแบบ แต่ที่นิยมกันในหมู่โปรแกรมเมอร์ที่เขียนโปรแกรมบนระบบปฏิบัติการวินโดว์ คือการกำหนดแบบฮังการี ซึ่งก็คือการกำหนดชื่อตัวแปรโดยใช้ตัวอักษรย่อของชนิดข้อมูลนำหน้าชื่อตัวแปร โดยมีการกำหนดดังตัวอย่าง

คำนำหน้า	ชนิดของข้อมูล
c	char, CHAR
by	BYTE
n	SHORT
i	int, INT
x, y	int, INT (x, y - coordinate)
cx,cy	int, INT (c - count)
b	BOOL
w	WORD
l	LONG
dw	DWORD
fn	function
str	string (STL)
sz	string terminated by 0 byte
h	HANDLE

p	พอยเตอร์
f	FLOAT
d3d	Direct3D Interface
lp	long พอยเตอร์

4.1.3 การเขียนโปรแกรมด้วย Win32 API

4.1.3.1 ฟังก์ชันจุดเริ่มต้น (The Entry-Point Function)

ในโปรแกรมที่มีพื้นฐานอยู่บน Win32 จะต้องมีฟังก์ชันจุดเริ่มต้น ซึ่งโดยทั่วไปจะมีชื่อว่า ฟังก์ชัน WinMain ซึ่งภายในฟังก์ชันนี้จะต้องประกอบด้วยขั้นตอนการทำงานดังนี้

4.1.3.2 การกำหนดค่าวินโดว์คลาส (Registering the window class)

ทุกๆ วินโดว์จะต้องมีวินโดว์คลาส (window class) ซึ่งเป็นตัวกำหนดคุณสมบัติ (attribute) ของวินโดว์ เช่น รูปแบบ (style), ไอคอน (icon), ชื่อของเมนู และชื่อของวินโดว์โปรซีเจอร์ (window procedure)

ขั้นตอนแรก คือ การใส่ค่าต่างๆ ในโครงสร้างข้อมูล WNDCLASS หรือ WNDCLASSEX ด้วยข้อมูลที่ต้องการ หลังจากนั้นจึงส่งค่าโครงสร้างข้อมูลที่ได้ใส่ค่าเรียบร้อยแล้วให้ฟังก์ชัน RegisterClass ในกรณีที่ใช้โครงสร้างข้อมูล WNDCLASS และฟังก์ชัน RegisterClassEx ในกรณีที่ใช้โครงสร้างข้อมูล WNDCLASSEX เช่น

```

WNDCLASSEX wcex;

wcex.cbSize          = sizeof(WNDCLASSEX);
wcex.style            = 0;
wcex.lpfnWndProc      = (WNDPROC)WndProc;
wcex.cbClsExtra       = 0;
wcex.cbWndExtra       = 0;
wcex.hInstance        = hInstance;
wcex.hIcon            = LoadIcon(hInstance, (LPCTSTR)IDI_MAIN);
wcex.hCursor          = LoadCursor(NULL, IDC_ARROW);
wcex.hbrBackground    = (HBRUSH)(COLOR_BTNFACE + 1);
wcex.lpszMenuName     = 0;
wcex.lpszClassName    = strClass;
wcex.hIconSm          = LoadIcon(wcex.hInstance, (LPCTSTR)IDI_SMALL);
RegisterClassEx(&wcex);

```

4.1.3.3 การสร้างวินโดว์หลัก (main window)

สามารถสร้างวินโดว์หลักได้ด้วยการเรียกฟังก์ชัน `CreateWindow` หรือ ฟังก์ชัน `CreateWindowEx` ดังนี้

```
hWnd = CreateWindowEx(    WS_EX_TOPMOST,
                          strClass, strName,
                          WS_OVERLAPPEDWINDOW,
                          CW_USEDEFAULT, CW_USEDEFAULT,
                          CW_USEDEFAULT, CW_USEDEFAULT,
                          NULL, NULL,
                          hInstance, NULL);
```

พารามิเตอร์ตัวที่สอง `strClass` คือ ชื่อของวินโดว์คลาสที่เราได้กำหนดเอาไว้แล้ว พารามิเตอร์อื่นๆ เป็นตัวกำหนดคุณสมบัติของวินโดว์ที่จะแสดงออกทางหน้าจอ แต่วินโดว์จะยังไม่แสดงออกหน้าจอจนกว่าจะเรียกฟังก์ชัน `ShowWindow` ดังนี้ `ShowWindow(m_hWnd, SW_NORMAL);`

4.1.3.4 การเข้าสู่เมสเสจลูป (Entering The Message Loop)

เมื่อสร้างวินโดว์หลักและสั่งให้แสดงผลออกทางหน้าจอแล้ว ฟังก์ชัน `WinMain` ก็จะเริ่มต้นทำหน้าที่หลักของมันคือ การออกเมสเสจจากแอปพลิเคชันคิว (application queue) และส่ง (dispatch) ให้กับวินโดว์ที่เหมาะสม

ระบบปฏิบัติการวินโดว์ไม่ได้ส่งข้อมูลให้กับโปรแกรมโดยตรง แต่จะส่งข้อมูลจากผู้ใช้ไว้ในเมสเสจคิว (message queue) เช่นเดียวกับเมสเสจที่ถูกส่งมาจากวินโดว์หรือโปรแกรมอื่นๆ โปรแกรมมีหน้าที่ที่จะต้องอ่านข้อมูลในเมสเสจคิว, รับเมสเสจเหล่านั้นไป และส่งให้กับวินโดว์โปรซีเดอร์ที่จะประมวลผลเมสเสจเหล่านั้นต่อไป

การเขียนฟังก์ชันที่ทำงานในเมสเสจลูปแบบทั่วไป

```
while(GetMessage(&msg, NULL, 0, 0))
{
    TranslateMessage(&msg);
    DispatchMessage(&msg);
}
```

`GetMessage` เป็นฟังก์ชันที่รับเมสเสจมาจากคิวและจะรอจนกว่าจะมีเมสเสจเข้ามาจึงจะส่งค่ากลับคืน (return)

`DispatchMessage` เป็นฟังก์ชันที่ส่งแต่ละเมสเสจไปยังวินโดว์โปรซีเดอร์ที่เหมาะสม

TranslateMessage เป็นฟังก์ชันที่แปลงเวอร์ชวลคีย์เมสเสจ (virtual-key message) ไปเป็นเมสเสจตัวอักษร ซึ่งมีความจำเป็นในการสร้างคีย์ในการเรียกเมนู (menu access key)

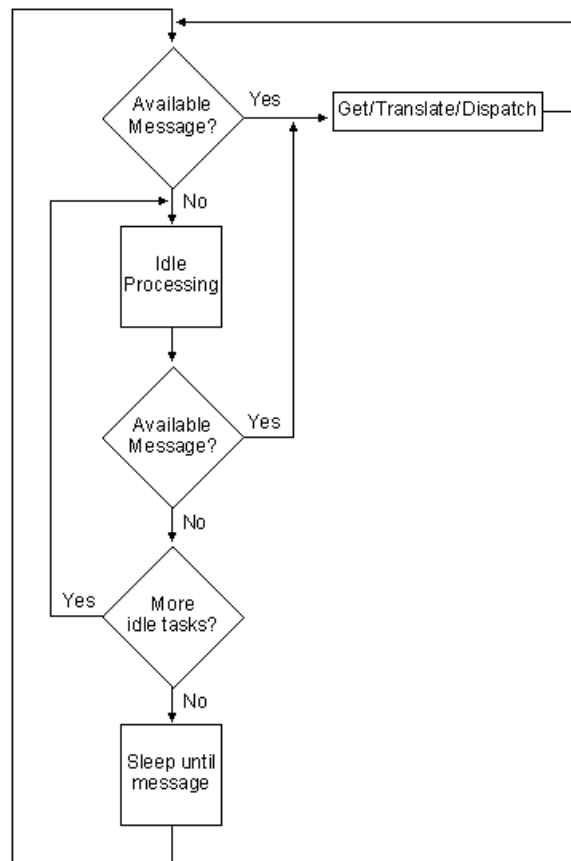
แต่โปรแกรมเกมโดยปกติแล้วจะทำงานตลอดเวลาเนื่องจากการเรนเดอร์ (render) จะต้องทำตลอดเวลาที่โปรแกรมทำงานอยู่แม้แต่ในเวลาที่ไม่มีเมสเสจของโปรแกรมเข้ามาด้วย ดังนั้น จึงไม่สามารถใช้ฟังก์ชัน GetMessage ได้ ต้องเขียนอีกวิธีหนึ่งดังนี้

4.1.3.5 การเขียนเมสเสจรูปแบบที่สามารถทำงานได้ตลอดเวลาสำหรับการเรนเดอร์

```
PeekMessage(&msg, NULL, 0U, 0U, PM_NOREMOVE);
```

```
while(WM_QUIT != msg.message)
{
    // Use PeekMessage() if the app is active, so we can use idle time to
    // render the scene. Else, use GetMessage() to avoid eating CPU time.
    if (bActive)
        bGotMsg = PeekMessage(&msg, NULL, 0U, 0U, PM_REMOVE);
    else
        bGotMsg = GetMessage(&msg, NULL, 0U, 0U);
    if (bGotMsg)
    {
        TranslateMessage(&msg);
        DispatchMessage(&msg);
    }
    else
    {
        // Render a frame during idle time (no messages are waiting)
    }
}
```

PeakMessage เป็นฟังก์ชันที่รับเมสเสจมาจากคิวแต่แตกต่างจากฟังก์ชัน GetMessage คือจะส่งค่ากลับคืน (return) มาทันทีโดยไม่รอว่ามีเมสเสจเข้ามาหรือไม่



รูปที่ 4-1 รูปแสดงขั้นตอนการทำงานในการเขียนเมสเสจลูปแบบทำงานตลอดเวลา

4.1.3.6 วินโดว์โปรซีเจอร์ (Window Procedure)

ทุกวินโดว์ (โปรแกรมหนึ่งๆ สามารถมีได้หลายวินโดว์) จะต้องมีวินโดว์โปรซีเจอร์ ซึ่งชื่อของวินโดว์โปรซีเจอร์สามารถตั้งได้ตามต้องการ (user-defined) ตัวอย่างของวินโดว์โปรซีเจอร์

```
LRESULT CALLBACK WndProc(HWND, UINT, WPARAM, LPARAM);
```

มีการใช้ตัวแปลง (modifier) CALLBACK เนื่องจากวินโดว์โปรซีเจอร์ต้องประกาศ (declare) โดยเรียกการเรียกอำนวยความสะดวก (calling convention) เสมอ

วินโดว์โปรซีเจอร์รับเมสเสจจากระบบปฏิบัติการวินโดว์ ซึ่งเมสเสจเหล่านั้นอาจจะเป็นข้อมูลที่รับเข้ามาจากผู้ใช้งานทางคีย์บอร์ดหรือเมาส์ หรืออาจจะเป็นสัญญาณควบคุมวินโดว์ (window-management message) ที่ส่งมาจากระบบปฏิบัติการวินโดว์ก็ได้ เราสามารถจัดการกับเมสเสจในวินโดว์โปรซีเจอร์ด้วยการเขียนโปรแกรมจัดการเองหรือส่งเมสเสจให้กับระบบปฏิบัติการวินโดว์เพื่อให้ประมวลผลแบบดีฟอลต์ (default processing) ได้ด้วยการเรียกฟังก์ชัน DefWindowProc ดังตัวอย่าง มีกำหนดวิธีประมวลผลเมื่อได้รับเมสเสจ WM_CREATE, WM_ACTIVE, และ WM_DESTROY โดยการใช้คำสั่ง switch ดังนี้

```

switch (uMsg)
{
    case WM_CREATE:
        // TO DO: handle create window message
        ShowWindow(hWnd,SW_NORMAL);
    break;
    case WM_ACTIVATE:
        // TO DO: handle activate window message
        dwActive = LOWORD(wParam);
        switch (dwActive)
        {
            case WA_ACTIVE:
                ShowWindow(hWnd,SW_RESTORE);
                bActive = TRUE;
                break;
            case WA_CLICKACTIVE:
                ShowWindow(hWnd,SW_RESTORE);
                bActive = TRUE;
                break;
            case WA_INACTIVE:
                ShowWindow(hWnd,SW_MINIMIZE);
                bActive = FALSE;
                break;
        }
    break;
    case WM_DESTROY:
        // TO DO: handle destroy window message
        PostQuitMessage(0);
    break;
    default:
        return DefWindowProc(hWnd, uMsg, wParam, lParam);
}

```


4.1.4 วินโดว์เมสเสจ (Message) ที่มีการใช้งานบ่อยๆ

เมสเสจที่ส่งมาจากคีย์บอร์ด (Keyboard Message)

WM_ACTIVATE	วินโดว์ถูกกระตุ้น
WM_KEYDOWN	กดแป้นคีย์บอร์ด
WM_KEYUP	ปล่อยแป้นคีย์บอร์ด
WM_CHAR	พิมพ์ตัวอักษร

เมสเสจที่ส่งมาจากระบบปฏิบัติการวินโดว์ (Windows Message)

WM_CREATE	สร้างวินโดว์
WM_COMMAND	คอมมานด์จากคอนโทรลต่างๆ เช่น เมนู เป็นต้น
WM_QUIT	ออกจากโปรแกรม
WM_CLOSE	ปิดโปรแกรม
WM_DESTROY	ทำลายวินโดว์
WM_SETTEXT	กำหนดตัวอักษรบนไทเทิลของวินโดว์
WM_MOVE	เคลื่อนย้ายวินโดว์
WM_SIZE	ย่อขยายขนาดของวินโดว์
WM_TIMER	ไทเมอร์ส่งค่ามาตามที่กำหนดไว้

เมสเสจที่ส่งมาจากเมาส์ (Mouse Message)

WM_MOUSEMOVE	เคลื่อนย้ายเมาส์เคอร์เซอร์
WM_MOUSEWHEEL	หมุนล้อที่อยู่บนเมาส์
WM_LBUTTONDOWNCLK	ดับเบิลคลิกเมาส์ปุ่มซ้าย
WM_LBUTTONDOWN	กดเมาส์ปุ่มซ้าย
WM_LBUTTONUP	ปล่อยเมาส์ปุ่มซ้าย
WM_RBUTTONDOWNCLK	ดับเบิลคลิกเมาส์ปุ่มขวา
WM_RBUTTONDOWN	กดเมาส์ปุ่มขวา
WM_RBUTTONUP	ปล่อยเมาส์ปุ่มขวา

4.1.5 การเลือกใช้ไทม์เมอร์ (Timer)

การใช้งานไทม์เมอร์เมสเสจ WM_TIMER มีประสิทธิภาพไม่เพียงพอที่จะนำมาใช้ในโปรแกรมเกม เนื่องจากมีโอเวอร์เฮด (overhead) ที่เกิดจากการทำงานในลักษณะของเมสเสจ นอกจากนี้ WM_TIMER ยังถูกกำหนดให้มีระดับความสำคัญ (priority) ต่ำกว่าวินโดว์เมสเสจอื่นๆ จำนวนมาก

นอกจาก การวัดค่าเวลาจากเมสเสจไทมเมอร์แล้ว เราสามารถเรียกฟังก์ชันเพื่ออ่านค่าเวลาในระหว่างแต่ละเมสเสจดูก็ได้ ซึ่งจะเป็นวิธีการอ่านค่าโดยตรงจากไทมเมอร์จึงไม่มีโอเวอร์เฮดเกิดขึ้น ไทมเมอร์ที่ใช้งานได้มี 2 ชนิด ดังนี้

1. ไทมเมอร์ความละเอียดสูง (high-resolution timer) สามารถอ่านค่าเวลาได้ด้วยการเรียกฟังก์ชัน `timeGetTime` ซึ่งไทมเมอร์ตัวนี้จะให้ความละเอียดในระดับ 1 มิลลิวินาที (1/1000 ของวินาที) ค่าที่ส่งคืนมาจะอยู่ในรูป `DWORD` ซึ่งไม่สามารถคำนวณหาวันเวลาจากค่านี้ได้ แต่สามารถนำไปใช้หาค่าเวลาที่เปลี่ยนแปลงไปได้โดยเก็บค่าเวลาเดิมไว้แล้วนำมาลบจากค่าที่อ่านได้
2. เคาทเตอร์ประสิทธิภาพสูง (performance counter) ในกรณีที่ต้องการค่าละเอียดมากยิ่งขึ้น เคาทเตอร์ประสิทธิภาพสูงซึ่งมีอยู่ในซีพียูรุ่นใหม่ๆ (ไม่ได้มีอยู่ในซีพียูทุกรุ่น) มีความละเอียดในระดับ 1 ไมโครวินาที (1/1000000 ของวินาที) ซึ่งโดยปกติแล้วเคาทเตอร์ในซีพียูจะทำงานที่ความถี่ 3.16 MHz แต่เนื่องจากไม่ได้มีอยู่ในซีพียูทุกรุ่นจึงต้องทำการตรวจสอบก่อนว่ามีอยู่หรือไม่โดยใช้ฟังก์ชัน `QueryPerformanceFrequency` ซึ่งโดยปกติจะคืนค่าความถี่ของเคาทเตอร์แต่ในกรณีที่ไม่มีเคาทเตอร์จะส่งคืนค่า 0 และใช้ฟังก์ชัน `QueryPerformanceCount` ในการอ่านค่าจากเคาทเตอร์

4.2 Standard Template Library (STL)

Standard Template Library คือ ชุดของคลาสและฟังก์ชัน (function) ที่ช่วยในการเก็บข้อมูล (collective of data) เช่น dynamic arrays (vector) และช่วยในการทำอัลกอริทึมง่าย เช่น การเรียงลำดับ (sorting) ซึ่งมีการกำหนดเป็นมาตรฐานโดย ANSI/ISO C++ Standards Committee ในปี 1994

STL ออกแบบมาโดยคำนึงถึงมาเร็วในการทำงานเป็นอันดับแรกจึงไม่ตรงกังวลเลยว่าการนำเอา STL มาใช้จะทำให้การทำงานของโปรแกรมที่เขียนขึ้นช้าลง

STL มีจำนวนมากและมีบางส่วนที่มีความซับซ้อน การใช้ STL ให้ได้อย่างมีประสิทธิภาพจึงต้องเข้าใจองค์ประกอบพื้นฐานและวิธีการใช้งานก่อนองค์ประกอบเหล่านี้ก่อน

4.2.1 องค์ประกอบพื้นฐานของ STL (Basic components of STL)

คอนเทนเนอร์ (Containers)

คอนเทนเนอร์ คือ STL template class ซึ่งจัดการลำดับในการเก็บข้อมูล ข้อมูลที่เก็บจะเป็นข้อมูลชนิดใดๆ ก็ได้ที่มี default constructor, destructor และ assignment operator ซึ่งแน่นอนว่าจะต้องมีความแตกต่างกันในการใช้งานคอนเทนเนอร์ในแต่ละชนิด

ไอเทอเรเตอร์ (Iterators)

ไอเทอเรเตอร์เปรียบเสมือนพอยเตอร์ที่ชี้ยังข้อมูลที่อยู่ในคอนเทนเนอร์ นอกจากนั้นแล้วใน STL ยังใช้สัญลักษณ์เดียวกับที่ใช้ในพอยเตอร์อีกด้วย เช่น ใน operator ++ ในการย้ายไอเทอเรเตอร์ไปยังข้อมูลตัวต่อไปที่อยู่ในคอนเทนเนอร์การย้ายตำแหน่งที่ชี้ข้อมูลของไอเทอเรเตอร์ไปยังข้อมูลตัวถัดไปในคอนเทนเนอร์

เรียกว่า increment นอกจากนั้น การเข้าถึงข้อมูลก็ใช้สัญลักษณ์ * เช่นเดียวกับพอยเตอร์ซึ่งเรียกการเข้าถึงข้อมูลนี้ว่า dereference

อัลกอริทึม (Algorithms)

อัลกอริทึม ที่ทำงานกับ STL ไม่ได้อยู่ในรูปของฟังก์ชันสมาชิกของคลาสคอนเทนเนอร์ แต่อยู่ในรูปของ stand-alone function ซึ่งทำงานอยู่บนไอน์เตอร์เรเตอร์ การแยกส่วนที่ทำหน้าที่ในการเก็บข้อมูลออกจากอัลกอริทึม ทำให้จำนวนของอัลกอริทึมที่ต้องเขียนขึ้นเฉพาะสำหรับแต่ละคอนเทนเนอร์ลงได้ เนื่องจากคอนเทนเนอร์ใดที่มีไอน์เตอร์เรเตอร์คล้ายคลึงกันก็สามารถใช้อัลกอริทึมร่วมกันได้

4.2.2 แนวความคิดของ STL (STL concepts)

หลักการต่อไปนี้เป็นสิ่งสำคัญในการใช้งาน STL สิ่งแรกก็คือ ต้องรู้ฟังก์ชันที่ให้หาช่วงของข้อมูล (จุดที่เริ่มเก็บข้อมูลถึงจุดสุดท้ายที่มีการเก็บข้อมูล) เมื่อใช้งานคอนเทนเนอร์การหาช่วงของข้อมูลมี 2 เมทอด (Method) ที่สำคัญ คือ begin() และ end() ด้วยการใส่ 2 เมทอดนี้สามารถหาช่วงของข้อมูลทั้งหมดที่มีอยู่ใน คอนเทนเนอร์ได้ โดย begin() จะส่งค่าตำแหน่งของข้อมูลตัวแรกในคอนเทนเนอร์และ end() จะส่งค่าตำแหน่งสุดท้ายของข้อมูลที่สามารถใช้งาน ข้อดีของการจัดการช่วงของข้อมูลในลักษณะนี้ คือ ไม่ต้องเขียนโค้ดมาจัดการกับช่วงของข้อมูลที่ไม่ได้ใช้งาน และสามารถนำไปใช้กับไอน์เตอร์เรเตอร์ที่ทำงานกับทั้งคอนเทนเนอร์ได้โดยการกำหนดให้ทำงานจนถึงค่าที่ส่งมาจากเมทอด end() ส่วนข้อเสีย ก็คือ การทำงานที่ไม่เป็นไปตามลำดับการทำงานของไอน์เตอร์เรเตอร์ต้องใช้ตัวแปรหรือไอน์เตอร์เรเตอร์พิเศษเข้าช่วย

นอกจากนั้นยังมีสิ่งที่ต้องคำนึงถึงอีกอย่างในการใช้งาน STL ก็คือ STL ออกแบบมาให้ส่งข้อมูลแบบ by value ไม่ใช่ by reference ดังนั้น ถ้าใช้งาน STL กับข้อมูลขนาดเล็กก็จะมีปัญหาแต่ถ้าจะนำไปใช้กับข้อมูลขนาดใหญ่ก็ควรใช้กับพอยเตอร์ของคลาสหรือโครงสร้างข้อมูล (struct) จะดีกว่า

4.2.3 คอนเทนเนอร์พื้นฐานใน STL ที่ใช้งานบ่อย

เวกเตอร์ (Vectors)

เวกเตอร์ใน STL หมายถึง อาร์เรย์ที่สามารถเปลี่ยนแปลงขนาดได้ (resizable array) ซึ่งแตกต่างจาก C++ อาร์เรย์ที่ไม่สามารถเปลี่ยนแปลงขนาดได้ อย่างไรก็ตาม มีความจำเป็นอย่างยิ่งที่จะต้องทำความเข้าใจวิธีการในการเปลี่ยนแปลงขนาดของเวกเตอร์ด้วย

เวกเตอร์สร้างขึ้นโดยใช้อาร์เรย์ (array) ดังนั้นในการเปลี่ยนแปลงขนาดจึงมีความจำเป็นที่จะต้องจองหน่วยความจำใหม่ (reallocate memory) แล้วจึงย้ายข้อมูลไปยังอาร์เรย์ใหม่ที่สร้างขึ้น หลังจากนั้นจึงปล่อยหน่วยความจำเดิม แต่เราก็สามารถจองพื้นที่หน่วยความจำให้มีขนาดใหญ่กว่าที่ต้องการใช้งานได้ เพื่อที่จะเหลือพื้นที่สำหรับในกรณีที่ต้องการขยายขนาดซึ่งจะทำให้การทำงานเร็วขึ้นเนื่องจากไม่ต้องจองหน่วยความจำและย้ายข้อมูลบ่อยๆ

ลิสต์ (Lists)

ลิสต์ใน STL สร้างขึ้นในแบบ doubly linked list ดังนั้น การแทรกหรือลบข้อมูลที่ใดๆ ก็ตามจึงสามารถทำได้ในเวลาอันรวดเร็ว แต่จะสูญเสียความสามารถในการเข้าถึงข้อมูลแบบสุ่ม (random access) ที่มีในเวกเตอร์และเดค (deque)

เดค (Dequeues)

เดค หรือ double-ended queue ได้รับการออกแบบมาสำหรับใช้ในกรณีที่ต้องการความสามารถในการแทรกหรือดึงข้อมูลออกจากปลายทั้งสองด้านของคอนเทนเนอร์ (ไม่มีความต้องการที่จะแทรกหรือดึงข้อมูลที่อยู่ตรงกลางออกแต่ก็สามารถทำได้) เช่นเดียวกับเวกเตอร์สามารถเข้าถึงข้อมูลในเดคแบบสุ่มได้ แต่ประสิทธิภาพจะต่ำกว่าการเข้าถึงข้อมูลในเวกเตอร์เนื่องจากความซับซ้อนของข้อมูลภายในของเดค ซึ่งเกิดจากการเรียงลำดับการเชื่อมต่อของ memory block

แมพ (Maps)

เป็น value-pairing container หมายความว่าข้อมูล 2 ชนิดที่จะต้องใช้คู่กันเสมอโดยมีโครงสร้างเป็น key/value แล้วจึงแทรกเข้าไปในคอนเทนเนอร์สามารถหาค่าในคอนเทนเนอร์คอนเทนเนอร์ได้โดยหาจากค่าของ key ซึ่งจะหาได้ในเวลา $O(\log n)$ ถึงแม้ว่าจะมีประสิทธิภาพไม่ดีเท่ากับการใช้ hash table แต่มีข้อดี เนื่องจากมีความสามารถในการเรียงลำดับข้อมูลระหว่างการแทรกข้อมูลอันเป็นผลมาจากวิธีการที่ใช้ในการเก็บข้อมูล (balanced binary tree หรือ red-black tree)

สแต็ก (Stacks)

สแต็กมีฟังก์ชันที่สำคัญ 3 ฟังก์ชัน คือ push(), pop() และ top() สำหรับใช้ในการเพิ่มและลบข้อมูลออกจากคอนเทนเนอร์โดยค่าเริ่มต้นแล้วสแต็กสร้างขึ้นโดยใช้เดคแต่เราสามารถเปลี่ยนชนิดของคอนเทนเนอร์ที่ใช้สร้างสแต็กได้โดยการกำหนดค่าคอนสตรัคเตอร์ (constructor) ดังนี้

```
// Implements a stack with deque as the underlying container type
```

```
stack<int> c;
```

```
// Implements a stack with a vector as the underlying container type
```

```
stack< int,vector<int> > c;
```

การใช้เวกเตอร์แทนการใช้เดคสามารถทำได้เนื่องจากสแต็กสามารถสร้างขึ้นจากคอนเทนเนอร์ใดๆ ก็ตามที่มีฟังก์ชัน push_back(), pop_back() และ back() เนื่องจากฟังก์ชัน push(), pop() และ top() ของสแต็กจะถูกจับเข้าคู่ (map) กับฟังก์ชันเหล่านี้ และข้อสำคัญอย่างลืมพิมพ์ช่องว่างระหว่างเครื่องหมาย > มิเช่นนั้นคอมไพเลอร์จะมองเป็น single stream operator >>

สิ่งสำคัญในการใช้ stack คือ อย่างลืมว่า STL คอนเทนเนอร์ส่วนใหญ่สร้างขึ้นโดยคำนึงถึงความเร็วเป็นอันดับแรก ดังนั้น ก่อนจะใช้คำสั่ง pop() และ top() ต้องแน่ใจว่ามีข้อมูลอยู่ในสแต็กโดยใช้ฟังก์ชัน size() หรือ empty() ในการตรวจสอบว่าสแต็กมีข้อมูลอยู่หรือไม่

คิว (Queues)

คิวทำงานคล้ายกับสแต็กยกเว้นข้อมูลจะถูกใส่จากด้านหลังและดึงออกได้จากด้านหน้า มีฟังก์ชันที่สำคัญดังนี้ push(), pop(), front() และ back() โดย back() ใช้อ้างอิงถึงข้อมูลที่เพิ่งเพิ่มเข้าไปในคิวและ front() ใช้อ้างอิงถึงข้อมูลที่อยู่ด้านหน้าสุดของคิวซึ่งจะถูกดึงออก

เช่นเดียวกับสแต็กนั้นก็ต้องใช้ฟังก์ชัน size() และ empty() ในการควบคุมขนาดของคิวนอกจากนั้นยังสามารถกำหนดชนิดของคอนเทนเนอร์ที่ใช้สร้างได้เช่นเดียวกับสแต็กด้วยแต่การใช้เวกเตอร์ในการสร้างคิวนั้นให้ประสิทธิภาพไม่ดีเนื่องจากการแทรกข้อมูลจากด้านหน้าของเวกเตอร์ไม่มีประสิทธิภาพนัก

4.2.4 การใช้งาน STL

Standard C++ library ทุกตัวถูกประกาศ (declare) หรือนิยาม (define) ไว้ใน standard header การใช้งานไลบรารี (library) เหล่านี้จึงต้อง include standard header ของ standard library ที่ต้องการใช้งานด้วย ซึ่ง STL ที่ใช้งานในโครงการนี้จะอยู่ใน standard header ดังนี้ <algorithm>, <list>, <map>, <queue>, <stack>, <string> และ <vector>

ตัวอย่างการ include standard header

```
// include vector facilities
```

```
#include <vector>
```

จะเห็นว่าการ include standard header จะแตกต่างจากการ include ไฟล์เฮดเดอร์ทั่วไปเนื่องจากไม่มี .h นอกจากนั้นการใช้งาน STL ควรจะประกาศ namespace ด้วย เนื่องจากชื่อทุกชื่อใน standard C++ header นิยามโดยใช้ std namespace ซึ่งถ้าไม่ประกาศ namespace แล้วการใช้งาน STL จะต้องเขียนโค้ดในรูปของ std::vector แต่ถ้าประกาศแล้วเขียนเพียงเวกเตอร์ก็สามารถใช้งานได้แล้ว ซึ่งการประกาศใช้ namespace ทำได้ดังนี้

```
using namespace std;
```